

Skill Week-4

Session-7

Task 1: Process Synchronization and Child Process Monitoring Using waitpid()

```
GNU nano 7.2 skill4s7task1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid, result;
    int status;

    printf("Parent Process Started. PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child Process Started. PID: %d\n", getpid());

        printf("Child is performing some work...\n");
        sleep(3);

        printf("Child Process Finished. PID: %d\n", getpid());
        exit(10);
    }
    else {
        // Parent process
        printf("Parent is waiting for Child PID: %d\n", pid);

        result = waitpid(pid, &status, 0);

        if (result == -1) {
            perror("waitpid failed");
            return 1;
        }
    }
}
```

```
GNU nano 7.2 skill4s7task1
    perror("fork failed");
    return 1;
}

if (pid == 0) {
    // Child process
    printf("Child Process Started. PID: %d\n", getpid());

    printf("Child is performing some work...\n");
    sleep(3);

    printf("Child Process Finished. PID: %d\n", getpid());
    exit(10);
}
else {
    // Parent process
    printf("Parent is waiting for Child PID: %d\n", pid);

    result = waitpid(pid, &status, 0);

    if (result == -1) {
        perror("waitpid failed");
        return 1;
    }

    if (WIFEXITED(status)) {
        printf("Child Process %d terminated normally.\n", result);
        printf("Child Exit Status: %d\n", WEXITSTATUS(status));
    }

    printf("Parent Process Finished. PID: %d\n", getpid());

    return 0;
}

^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste    ^J Justify
```

```

This message is shown once a day. To disable it please create the
/home/ruhi0404/.hushlogin file.
ruhi0404@Ruhitha:~$ cd ~/Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ pwd
/home/ruhi0404/Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ nano skill4s7task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill4s7task1.c -o skill4s7task1
ruhi0404@Ruhitha:~/Operating_Systems$ ls
File1.txt  bin          pra2task2  prac1task1.c  skill      skill2task2.c  skill4s7task1.c  skills4task1.c  skills5task2.c  skillw3task2.c
File2.txt  docs        pra3task1  prac2task1.c  skill1task2  skill2w1task1  skill53task2     skills4task2     skills6task1     src
LICENSE   include     pra3task1.c  prac2task1.c  skill1task2.c  skill2w1task1.c  skill53task2.c  skills4task2.c  skills6task1.c  tests
Makefile   obj         pra3task2  practical     skill2task1  skill2w1task2  skill53week2     skills5task1     skills6task2
README.md  pra1task1  pra3task2.c  sample.txt    skill2task1.c  skill2w1task2.c  skill53week2.c  skills5task1.c  skills6task2.c
assets     pra1task1.c  prac1task1  scripts       skill2task2  skill4s7task1  skill54task1     skill55task2     skillw3task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill4s7task1
Parent Process Started. PID: 637
Parent is waiting for Child PID: 638
Child Process Started. PID: 638
Child is performing some work...
Child Process Finished. PID: 638
Child Process 638 terminated normally.
Child Exit Status: 10
Parent Process Finished. PID: 637
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully demonstrates process synchronization and child process monitoring using `waitpid()`. A child process is created using `fork()`, and the parent process waits for the specific child to complete using `waitpid()`. The child performs its task and terminates with an exit status of 10. The parent then detects the child's normal termination using `WIFEXITED()` and retrieves its exit status using `WEXITSTATUS()`. Thus, `waitpid()` ensures that the parent process does not continue until the child process has completed, demonstrating proper synchronization between parent and child processes.

Task 2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

Function	Purpose
<code>getenv()</code>	Retrieves the PATH environment variable
<code>strtok()</code>	Splits PATH into individual directories
<code>access()</code>	Checks file existence and execute permission
<code>snprintf()</code>	Builds the complete executable path

strdup()	Creates a modifiable copy of PATH
free()	Releases allocated memory

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main() {
    char *path;
    char *path_copy;
    char *directory;
    char full_path[1024];
    char command[100];

    // Retrieve PATH environment variable
    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH environment variable not found.\n");
        return 1;
    }

    // Get command from user
    printf("Enter command to locate: ");
    scanf("%99s", command);

    // Create a modifiable copy of PATH
    path_copy = strdup(path);

    if (path_copy == NULL) {
        perror("strdup failed");
        return 1;
    }

    // Split PATH into individual directories
    directory = strtok(path_copy, ":");

    while (directory != NULL) {
```

```
scanf("%99s", command);

// Create a modifiable copy of PATH
path_copy = strdup(path);

if (path_copy == NULL) {
    perror("strdup failed");
    return 1;
}

// Split PATH into individual directories
directory = strtok(path_copy, ":");

while (directory != NULL) {
    // Build complete executable path
    snprintf(full_path, sizeof(full_path), "%s/%s", directory, command);

    // Check if file exists and has execute permission
    if (access(full_path, X_OK) == 0) {
        printf("Executable found: %s\n", full_path);
        free(path_copy);
        return 0;
    }

    directory = strtok(NULL, ":");
}

// Command was not found
printf("Command '%s' not found in PATH.\n", command);

// Release allocated memory
free(path_copy);

return 0;
}

G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_/_ G
```

```
Parent Process Finished. PID: 637
ruhi0404@Ruhitha:~/Operating_Systems$ nano skill4s7task2.c
ruhi0404@Ruhitha:~/Operating_Systems$
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill4s7task2.c -o skill4s7task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill4s7task2
Enter command to locate: ls
Executable found: /usr/bin/ls
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill4s7task2
Enter command to locate: gcc
Executable found: /usr/bin/gcc
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill4s7task2
Enter command to locate: hello777
Command 'hello777' not found in PATH.
ruhi0404@Ruhitha:~/Operating_Systems$ |
```

Observation

The program successfully retrieves the PATH environment variable using `getenv()` and creates a modifiable copy using `strdup()`. The `strtok()` function is used to separate the PATH into individual directories, while `snprintf()` constructs the complete path of the entered command. The `access()` function verifies whether the executable exists and has execute permission. The program correctly displays the executable location for valid commands and reports an appropriate message when the command is not found. Finally, `free()` releases the dynamically allocated memory, demonstrating proper memory management.

Session 8

Task 1: To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_OUTPUT 2048

void expand_variables(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1) {

        // Detect variable reference
        if (input[i] == '$') {
            i++;

            // Handle ${VARIABLE} format
            if (input[i] == '{') {
                i++;

                char variable[100];
                int k = 0;

                while (input[i] != '\0' &&
                    input[i] != '}' &&
                    k < 99) {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                if (input[i] == '}')
                    i++;

                char *value = getenv(variable);
```


Task 2:

:To Identify Built-ins, Create Dispatch Table, Execute In-Process Built-ins, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Function	Purpose
chdir()	Changes the current working directory
getcwd()	Gets the current working directory
getenv()	Retrieves environment variables
exit()	Terminates the process
strcmp()	Compares command names
strtok()	Separates command into tokens

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void execute_builtin(char *command, char *argument)
{
    if (strcmp(command, "cd") == 0)
    {
        if (argument == NULL)
        {
            printf("cd: missing argument\n");
        }
        else if (chdir(argument) != 0)
        {
            perror("cd");
        }
    }

    else if (strcmp(command, "pwd") == 0)
    {
        char cwd[1024];

        if (getcwd(cwd, sizeof(cwd)) != NULL)
            printf("%s\n", cwd);
        else
            perror("getcwd");
    }

    else if (strcmp(command, "getenv") == 0)
    {
        if (argument == NULL)
        {
            printf("getenv: missing variable name\n");
        }
        else
        {
            // Implementation for getenv command
        }
    }
}
```



```

        printf("getenv: missing variable name\n");
    }
    else
    {
        char *value = getenv(argument);

        if (value != NULL)
            printf("%s = %s\n", argument, value);
        else
            printf("Environment variable not found\n");
    }
}

else if (strcmp(command, "exit") == 0)
{
    printf("Exiting shell...\n");
    exit(0);
}

else
{
    printf("Invalid command: %s\n", command);
}
}

int main()
{
    char input[100];
    char *command;
    char *argument;

    printf("Mini Built-in Shell\n");
    printf("Commands: cd, pwd, getenv, exit\n");

    while (1)
    {
        printf("myshell> ");

```

```

GNU nano 7.2
    }

    else
    {
        printf("Invalid command: %s\n", command);
    }
}

int main()
{
    char input[100];
    char *command;
    char *argument;

    printf("Mini Built-in Shell\n");
    printf("Commands: cd, pwd, getenv, exit\n");

    while (1)
    {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        command = strtok(input, " ");
        argument = strtok(NULL, " ");

        if (command != NULL)
        {
            execute_builtin(command, argument);
        }
    }

    return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skills8ttask2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skills8ttask2.c -o skills8ttask2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skills8ttask2
Mini Built-in Shell
Commands: cd, pwd, getenv, exit
myshell> pwd
/home/ruhi0404/Operating_Systems
myshell> cd /tmp
myshell> pwd
/tmp
myshell> getenv HOME
HOME = /home/ruhi0404
myshell> getenv PATH
PATH = /usr/local/bin:/usr/bin:/usr/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/Microsoft
corporationII.WindowsSubsystemForLinux.2.7.18_0_x64_8wekyb3d8bbwe:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common F
iles/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/WINDOWS/System32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:
/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/b/Program Files/nodejs:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Fi
les/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/Lenovo/AppData/Local/Programs/Python/Launcher:/mnt/c/Users/Lenovo/.local/bin:/mnt/c/Users/Lenovo/AppData/Local/
Microsoft/WindowsApps:/mnt/c/Users/Lenovo/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/Lenovo/AppData/Roaming/npm:/mnt/b/Program Files/JetBrain
s/PyCharm 2026.1.2/bin:/mnt/c/ProgramData/Lenovo/GitHubDesktop/bin:/snap/bin
myshell> hello
Invalid command: hello
myshell> exit
Exiting shell...
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation:

The program successfully implements a simple built-in command dispatcher. The `strcmp()` function identifies the entered command, while `strtok()` separates the command and its argument. The `chdir()` function changes the current directory, `getcwd()` displays the current working directory, `getenv()` retrieves environment variables, and `exit()` terminates the program. Invalid commands are detected and an appropriate error message is displayed. The program maintains the current directory state between commands.